

THE UNIVERSITY OF AUCKLAND

Semester One, 2026

Campus: City

Software Engineering Operating Systems

(Time Allowed: ONE hour)

NOTE:

- This exam is closed book.
- ONE double-sided A4 cheat sheet is allowed.
- The use of calculators or other devices is NOT permitted.
- Compare the exam version number on the Teleform sheet supplied with the version number above. If they do not match, ask the supervisor for a new sheet.
- Enter your name and student ID on the Teleform sheet. Your name should be entered left aligned. If your name is longer than the number of boxes provided, truncate it.
- Answer ALL multiple-choice questions on the Teleform answer sheet provided.
- Use a dark pencil to mark your answers in the multiple-choice answer boxes on the Teleform sheet. Check that the question number on the sheet corresponds to the question number in this question book. If you spoil your sheet, ask the supervisor for a replacement.
- Choose the BEST answer for each question based on the material covered in the course.
- There are 30 questions in this exam, worth 30 marks in total. Allocate your time appropriately.

Surname:

First Name(s):

Student ID:

Question 1 [1 mark]

Consider the C program below. Which of the following statements is true?

```
int *make(void) {
    int z = 11;
    int *p = &z;
    return p;
}
int main(void) {
    int *q = make();
    printf("%d\n", *q);
    return 0;
}
```

- A. The function `make` has undefined behavior as soon as it returns `p`.
- B. The declaration of `z` is illegal because its address escapes the block.
- C. The behavior depends only on the calling convention.
- D. The program is well-defined, but the printed value is always garbage.
- E. The call to `printf()` has undefined behavior.

Question 2 [1 mark]

Consider the C program below. Which of the following statements are true?

```
int g;
int main(void) {
    int l;
    int *p = malloc(sizeof(int));
    ...
    free(p);
    return 0;
}
```

- (1) `&g`, `&l`, and `p` must all be virtual addresses.
 - (2) `&g` is a physical address because a global variable is placed in a fixed memory region for the whole execution of the program.
 - (3) Under some special conditions, `&l` and `p` can be physical addresses because stack and heap memory may be backed directly by RAM after allocation.
- A. (1) and (2)
 - B. (1)
 - C. All the statements are true.
 - D. (1) and (3)
 - E. (2) and (3)

Question 3 [1 mark]

Which of the following statements are true?

- (1) A process's virtual address space may be larger than physical memory.
- (2) Every valid virtual address of a process must always correspond to a location currently in physical memory.
- (3) The same virtual address in two different processes can refer to the same physical memory location.

- A. (1) true; (2) true; (3) false
- B. (1) true; (2) false; (3) true
- C. (1) true; (2) false; (3) false
- D. (1) false; (2) false; (3) true
- E. (1) false; (2) true; (3) true

Question 4 [1 mark]

Consider the C program below. Suppose this program is started twice as two separate running processes, with process 1 starting slightly before process 2. Which of the following statements are true?

```
int g = 0;
int main(void) {
    int l = 0;
    while(1) {
        g++;
        l++;
        printf("%d %d\n", g, l);
    }
    return 0;
}
```

- (1) In each instance, `l` is created separately when `main` runs.
- (2) A change to `l` in one instance can change `l` in the other instance.
- (3) A change to `g` in one instance can change `g` in the other instance.

- A. (1) true; (2) true; (3) false
- B. (1) true; (2) false; (3) false
- C. (1) false; (2) false; (3) true
- D. (1) false; (2) false; (3) false
- E. (1) true; (2) false; (3) true

Question 5 [1 mark]

Which of the following statements are true?

- (1) A system call always transfers execution from user mode to kernel mode.
- (2) A system call is simply an ordinary function call into kernel code, so its arguments must be passed by a pointer to kernel memory.
- (3) The kernel changes the mode bit when a system call is executed, transferring control from user mode to kernel mode.

- A. (1) false; (2) true; (3) true
- B. (1) false; (2) false; (3) true
- C. (1) true; (2) false; (3) false
- D. (1) true; (2) false; (3) true
- E. (1) false; (2) true; (3) false

Question 6 [1 mark]

For each design scenario below, choose the more appropriate kernel design.

- (1) An operating system is being designed for a performance-critical environment where major services should interact with as little overhead as possible.
- (2) An operating system is being designed so that faults in services such as device drivers or file systems are less likely to compromise the whole system.
- (3) An operating system is being designed so that core system services are tightly integrated within the privileged part of the operating system.

- A. (1) Microkernel, (2) Monolithic, (3) Microkernel
- B. (1) Monolithic, (2) Microkernel, (3) Monolithic
- C. (1) Monolithic, (2) Microkernel, (3) Microkernel
- D. (1) Microkernel, (2) Microkernel, (3) Monolithic
- E. (1) Monolithic, (2) Monolithic, (3) Microkernel

Question 7 [1 mark]

Consider the C program below. What is the output?

```
int main(void) {  
    printf("14");  
    sleep(1);  
    write(1, "23", 2);  
    return 0;  
}
```

- A. 1243
- B. 2314
- C. The output is not uniquely determined; more than one of the above outputs is possible.
- D. 1423
- E. 1234

Question 8 [1 mark]

Consider the C program below. Which of the following outputs *cannot* occur?

```
int main(void) {  
    if (fork() == 0) {  
        printf("B");  
        exit(0);  
    }  
    if (fork() == 0) {  
        printf("C");  
        exit(0);  
    }  
    wait(NULL);  
    printf("A");  
    return 0;  
}
```

- A. CAB
- B. ABC
- C. BCA
- D. More than one of these three outputs is impossible
- E. All these three outputs are possible

Question 9 [1 mark]

Consider the following heap manager design:

- i. The allocator gets memory from the OS in 2 KiB pages.
- ii. It uses a first-fit policy, automatically splits a free block if it is larger than a request, and automatically coalesces a block with its neighbouring free blocks when it is freed.
- iii. The heap address grows from small to large, and if a block is split, the lower-address portion is returned to the user.

Consider the following call sequence (heap is initially empty). Which statement is true about the heap after the calls finish executing?

- A. The heap has a size of 4 KiB with one free block.
- B. The heap has a size of 4 KiB with two free blocks.
- C. The heap has a size of 2 KiB with two free blocks.
- D. The heap has a size of 2 KiB with one free block.
- E. The heap has a size of 2 KiB with zero free block.

Question 10 [1 mark]

Consider the C program below, compiled as `./a.out`. Assume `fork()` and `execl()` always succeed. How many processes will exist after it has been running for some time?

```
int main(void) {  
    fork();  
    execl("./a.out", "./a.out", NULL);  
    fork();  
    return 0;  
}
```

- A. 1
- B. 2
- C. 4
- D. None of the given options
- E. 8

Question 11 [1 mark]

A text editor process has just been started by the user. It is loaded by the OS, becomes eligible to run, begins executing on the CPU, then requests input from the keyboard and must wait for the user to type. After the user types a key, the process becomes eligible to run again, later gets CPU time, and finally exits.

Which sequence of process states best matches this behavior?

- A. Created → Running → Ready → Blocked → Running → Terminated
- B. Created → Ready → Running → Blocked → Ready → Running → Terminated
- C. Created → Running → Blocked → Ready → Terminated
- D. Created → Ready → Blocked → Running → Ready → Terminated
- E. Created → Blocked → Running → Ready → Running → Terminated

Question 12 [1 mark]

Consider the C program below. What is the output of the program?

```
int main(void) {  
    int p = 0;  
    pid_t parent = getpid();  
    fork();  
    p++;  
    fork();  
    p++;  
    if (getpid() == parent)  
        printf("%d\n", p);  
    return 0;  
}
```

- A. The output is not deterministic; it depends on the scheduling of the parent and child processes.
- B. 2
- C. 0
- D. 6
- E. 4

Question 13 [1 mark]

Consider the C program below. What is the output of the program, excluding the newline characters?

```
int main(void) {
    int p[2];
    char x;
    pipe(p);
    if (fork() == 0) {
        read(p[0], &x, 1);
        printf("B\n");
        exit(0);
    }
    printf("A\n");
    write(p[1], "x", 1);
    wait(NULL);
    printf("C\n");
    return 0;
}
```

- A. More than one of these outputs is possible
- B. ABC
- C. BCA
- D. BAC
- E. ACB

There are 4 processes in your system. Only 1 process can run at a time.

P1, arrival time = 0, burst time = 3

P2, arrival time = 1, burst time = 6

P3, arrival time = 2, burst time = 3

P4, arrival time = 2 (arrives after P3), burst time = 1

If a process arrives at time 0, requests a time slice of 1 unit, and is executed immediately, it will complete at time 1. Use FCFS for all tiebreaks except Round Robin. *Answer the next 5 questions.*

Question 14 [1 mark]

What is the completion time for all four processes under First Come First Serve (FCFS)?

- A. P1:4, P2:10, P3:13, P4:11
- B. P1:3, P2:9, P3:13, P4:10
- C. P1:3, P2:12, P3:9, P4:10
- D. P1:3, P2:10, P3:12, P4:13
- E. P1:3, P2:9, P3:12, P4:13

Question 15 [1 mark]

What is the completion time for all four processes under Shortest Job First?

- A. P1:4, P2:13, P3:7, P4:5
- B. P1:3, P2:9, P3:12, P4:13
- C. P1:3, P2:13, P3:7, P4:4
- D. P1:3, P2:10, P3:13, P4:4
- E. P1:3, P2:13, P3:10, P4:4

Question 16 [1 mark]

What is the completion time for all four processes under Shortest Remaining Time First (a.k.a Shortest Job-to-Completion First)?

- A. P1:6, P2:13, P3:9, P4:4
- B. P1:3, P2:12, P3:9, P4:4
- C. P1:6, P2:13, P3:7, P4:4
- D. P1:3, P2:13, P3:7, P4:4
- E. P1:4, P2:13, P3:7, P4:3

Question 17 [1 mark]

What is the completion time for all four processes under Round Robin with time slice of 2?

- A. P1:7, P2:13, P3:11, P4:8
- B. P1:3, P2:12, P3:13, P4:8
- C. P1:8, P2:12, P3:11, P4:7
- D. P1:8, P2:13, P3:11, P4:7
- E. P1:8, P2:11, P3:13, P4:7

Question 18 [1 mark]

After each process finishes its current CPU burst, it may later return to the ready queue and request another CPU burst of the same length as above. The time at which each process returns is unpredictable, and this may continue indefinitely. Which scheduling algorithms can guarantee that a process that is ready to run will eventually be scheduled?

- A. FCFS only
- B. Round Robin only
- C. FCFS and Round Robin only
- D. SRTF only
- E. SJF only

Question 19 [1 mark]

A company is building software for a multi-core scientific simulation. The application launches a fixed number of long-running worker processes at startup. During each phase, a worker repeatedly reuses much of the same data from recent computations. Each worker repeatedly performs similar computations throughout the run, and the problem has been divided so that the workers are expected to remain similarly busy for most of the execution. They are considering three scheduler designs:

Design A: one global ready queue shared by all CPUs

Design B: one ready queue per CPU, with no movement between CPUs

Design C: one ready queue per CPU, with frequent movement of work between CPUs

Which design is the most appropriate for this scenario?

- A. Design A or C are equally suitable
- B. Design A
- C. Design C
- D. None of the above
- E. Design B

Consider a system with a 30-bit virtual address, a 512-byte page size, and a PTE size of 8 bytes. For a multi-level page table, assume each page table must fit within one physical frame. Answer the next 5 questions.

Question 20 [1 mark]

For a single-level page table, how many PTEs are there?

- A. 2^{21}
- B. 2^{20}
- C. 2^{23}
- D. 2^{24}
- E. 2^{22}

Question 21 [1 mark]

For a multi-level page table, what is the size of each page table?

- A. 4 KiB
- B. 256 Bytes
- C. 512 Bytes
- D. 1 KiB
- E. 2 KiB

Question 22 [1 mark]

For a multi-level page table, what is the minimum number of page-table levels required?

- A. 4
- B. 3
- C. 6
- D. 2
- E. 5

Question 23 [1 mark]

Since the beginning of execution, your program has repeatedly accessed the following addresses: 0x08418812, 0x08418A1A, 0x0842021F. How many bytes of the page table have been allocated so far?

- A. 4096 Bytes
- B. 3584 Bytes
- C. 4608 Bytes
- D. 2560 Bytes
- E. 3072 Bytes

Question 24 [1 mark]

Your program maps exactly 10 data pages (code + data). What are the minimum and maximum number of pages allocated in total?

- A. 13, 41
- B. 13, 31
- C. 14, 41
- D. 14, 39
- E. 13, 21

Question 25 [1 mark]

Consider the C program below. What is the output of the program?

```
struct S {  
    char a; char b; int c;  
    char d; int e; char f;  
};  
int main(void) {  
    printf("%lu", sizeof(struct S));  
    return 0;  
}
```

- A. 19
- B. 18
- C. 17
- D. 20
- E. 16

Question 26 [1 mark]

You are tasked with improving the memory system of a processor. Initially, the Translation Lookaside Buffer (TLB) access time is 20% of the memory access time. A redesign halves the TLB access time, but the TLB miss rate increases by 10 percentage points (e.g., 30% to 40%). The TLB size remains unchanged. Your system uses a single-level page table. What is the overall change in effective access time?

- A. -5%
- B. 5%
- C. 10%
- D. -10%
- E. It stays the same.

Question 27 [1 mark]

A processor includes a Translation Lookaside Buffer (TLB) to speed up address translation during virtual memory access. Which of the following best describes what the TLB stores?

- A. Recent address translations
- B. Entire page table levels
- C. MMU control instructions
- D. Frequently used page contents
- E. Recently loaded physical pages

Question 28 [1 mark]

Using a buddy allocator with a 256-byte memory space, consider the following requests in order: 28, 90, 14, 6. Assuming allocated blocks require no headers, how many free blocks of exactly 16 bytes exist after all requests are complete?

- A. 3
- B. 1
- C. 0
- D. 4
- E. 2

Question 29 [1 mark]

Which of the following statements about `malloc()` are correct?

- (1) If `malloc()` returns a non-NULL pointer, physical memory is guaranteed to have already been allocated for the entire block.
- (2) If `malloc()` returns a non-NULL pointer, writing to every byte in the allocated block is guaranteed to succeed.
- (3) The memory returned by `malloc()` remains allocated until it is freed with `free()`, even after the process terminates.

- A. (1) true; (2) false; (3) false
- B. (1) true; (2) true; (3) true
- C. (1) false; (2) false; (3) true
- D. (1) false; (2) false; (3) false
- E. (1) false; (2) true; (3) true

Question 30 [1 mark]

How many of the following statements about x86 virtual memory are true?

- (1) A TLB miss during a memory access does not always mean the access will trigger a page fault.
- (2) Memory loads and stores do not always invoke the OS kernel.
- (3) The operating system determines which pages should be replaced on x86.
- (4) Switching from a single-level to a multi-level page table does not inherently change how often TLB misses occur.

- A. 4
- B. 3
- C. 1
- D. 0
- E. 2

Question 31 [Bonus Question]

What is the class average on this test, excluding this bonus question?

- A. <60%
- B. 60-70%
- C. 70-80%
- D. 80-90%
- E. 90-100%

STUDENT NAME: _____

STUDENT ID

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9

SCRIPT VERSION

0	0	0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9



Waipapa
Taumata Rau
University
of Auckland

INSTRUCTIONS

Use an erasable soft pencil.
Fill the bubbles completely.
Erase completely to change.
Student ID alignment does not matter.
Do not fold this form or write on the back.

RESPONSES

1)	A	B	C	D	E	F	21)	A	B	C	D	E	F	41)	A	B	C	D	E	F	61)	A	B	C	D	E	F
2)	A	B	C	D	E	F	22)	A	B	C	D	E	F	42)	A	B	C	D	E	F	62)	A	B	C	D	E	F
3)	A	B	C	D	E	F	23)	A	B	C	D	E	F	43)	A	B	C	D	E	F	63)	A	B	C	D	E	F
4)	A	B	C	D	E	F	24)	A	B	C	D	E	F	44)	A	B	C	D	E	F	64)	A	B	C	D	E	F
5)	A	B	C	D	E	F	25)	A	B	C	D	E	F	45)	A	B	C	D	E	F	65)	A	B	C	D	E	F
6)	A	B	C	D	E	F	26)	A	B	C	D	E	F	46)	A	B	C	D	E	F	66)	A	B	C	D	E	F
7)	A	B	C	D	E	F	27)	A	B	C	D	E	F	47)	A	B	C	D	E	F	67)	A	B	C	D	E	F
8)	A	B	C	D	E	F	28)	A	B	C	D	E	F	48)	A	B	C	D	E	F	68)	A	B	C	D	E	F
9)	A	B	C	D	E	F	29)	A	B	C	D	E	F	49)	A	B	C	D	E	F	69)	A	B	C	D	E	F
10)	A	B	C	D	E	F	30)	A	B	C	D	E	F	50)	A	B	C	D	E	F	70)	A	B	C	D	E	F
11)	A	B	C	D	E	F	31)	A	B	C	D	E	F	51)	A	B	C	D	E	F	71)	A	B	C	D	E	F
12)	A	B	C	D	E	F	32)	A	B	C	D	E	F	52)	A	B	C	D	E	F	72)	A	B	C	D	E	F
13)	A	B	C	D	E	F	33)	A	B	C	D	E	F	53)	A	B	C	D	E	F	73)	A	B	C	D	E	F
14)	A	B	C	D	E	F	34)	A	B	C	D	E	F	54)	A	B	C	D	E	F	74)	A	B	C	D	E	F
15)	A	B	C	D	E	F	35)	A	B	C	D	E	F	55)	A	B	C	D	E	F	75)	A	B	C	D	E	F
16)	A	B	C	D	E	F	36)	A	B	C	D	E	F	56)	A	B	C	D	E	F	76)	A	B	C	D	E	F
17)	A	B	C	D	E	F	37)	A	B	C	D	E	F	57)	A	B	C	D	E	F	77)	A	B	C	D	E	F
18)	A	B	C	D	E	F	38)	A	B	C	D	E	F	58)	A	B	C	D	E	F	78)	A	B	C	D	E	F
19)	A	B	C	D	E	F	39)	A	B	C	D	E	F	59)	A	B	C	D	E	F	79)	A	B	C	D	E	F
20)	A	B	C	D	E	F	40)	A	B	C	D	E	F	60)	A	B	C	D	E	F	80)	A	B	C	D	E	F